ShiaanX — Developer Onboarding Guide

> ***Internal document. Do not distribute externally.*** *Last updated: July 2026*

---

1. What We Are Building

ShiaanX is building an AI-driven process planning platform for precision manufacturing. The core product takes a CAD file as input and produces everything a machine shop needs to make the part — without a human engineer deciding the process at every step.

The problem we are solving: getting from a 3D CAD model to a correctly-machined part currently requires experienced engineers making hundreds of decisions (machine selection, tooling, workholding, feeds and speeds, setup sequence). These decisions live largely in people's heads. ShiaanX systematises them.

Current focus: 3-axis CNC milling of aluminium parts, targeting precision industries (aerospace, defence, drones).

What we are not building: a CAM software, a job board, or a generic quoting tool. We sit above the CAM layer and compete on engineering depth and traceability, not price.

---

2. Current Status and Roadmap

Working end-to-end

- Full pipeline from STEP upload to PDF program sheet
- 3D viewer with part geometry
- Fixture planning with soft jaw generation for vise setups
- Per-setup workholding configuration and WCS
- Frontend with all viewer tabs (some tabs are stubs pending backend data)

Active development (branch: `feature/fixture-viewer`)

- 3D fixture assembly tab — part + soft jaws rendered together
- Feature map tab — per-face feature annotation in the viewer
- Datum tagger — annotate datum reference frames on the 3D model

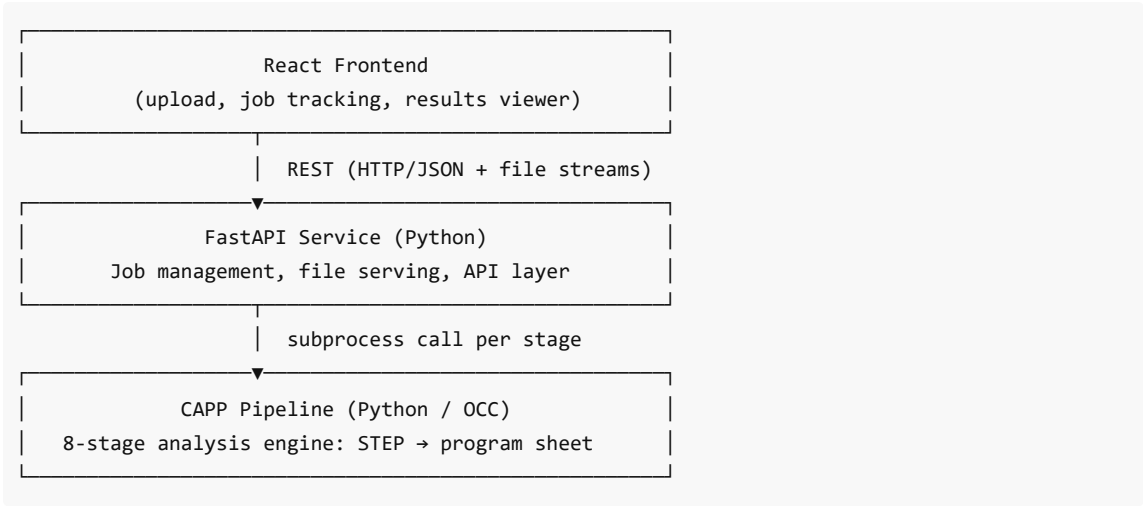
Roadmap (prioritised)

Priority	Work item	Notes
1	Inspection checklist	Per-feature, built on top of datum reference frames — depends on datum tagger being complete first
2	Closed-loop feedback capture	Connecting the capture UI to actual data storage and loop-back
3	AI model improvements	Feature recognition, setup planning, tool selection, parameters — ongoing, highest engineering value
4	CAM integration (Fusion 360)	Add-in exists in `Toolpath/`; needs wiring to CAPP service and operation-level import

---

3. System Overview

The system has three distinct layers:



A user uploads a STEP file. The service spawns the pipeline as a background job. The frontend polls for status and progressively displays results as each stage completes.

---

4. Tech Stack

Layer	Technology
Frontend	React 19, React Router v7, Axios
Backend	Python 3.10, FastAPI, Uvicorn
Pipeline	Python, PythonOCC (OpenCASCADE bindings)
3D Viewer	Embedded iframe (CAD viewer via URL)
PDF generation	ReportLab
Python environment	Conda (`occ` environment)
CNC telemetry store	InfluxDB (time series — controller data)
Business data	PostgreSQL (orders, enquiries, parts)
File storage	AWS S3 (CAD files, drawings, PDFs)
CAM integration	Fusion 360 Python add-in (`Toolpath/`)

---

5. Repository Layout

```
ShiaanX/
├── capp-frontend/          # React app
│   └── src/
│       ├── pages/          # Upload, CappViewer
│       ├── components/     # Layout, viewer tabs
│       └── services/       # API client (axios)
```

```
│
├── capp_service/           # FastAPI backend
│   ├── main.py             # All API routes
│   ├── runner.py           # Spawns pipeline subprocess
│   └── jobs/               # Per-job working directories (gitignored)
│
├── Toolpath/               # Fusion 360 add-in (CAM integration)
│   ├── Toolpath.py         # Add-in entry point
│   ├── code/
│   │   ├── commands/       # Individual add-in commands
│   │   └── lib/            # Fusion API utilities, HTTP client
│   └── Toolpath.manifest   # Add-in metadata for Fusion
│
└── Claude output for program sheet/   # CAPP pipeline engine
    ├── 1. extract_features.py
    ├── 2. cluster_features.py
    ├── 3. classify_features.py
    ├── 4. process_selection.py
    ├── 5. setup_planning.py
    ├── 7. tool_selection.py
    ├── 8. parameter_calculation.py
    ├── 9. program_sheet.py
    ├── 10. run_pipeline.py         # End-to-end runner
    ├── 7a. tool_database.json      # Tooling data
    ├── fixture_design/             # Soft jaw + fixture planning module
    └── rule_sheets/                # Validated machining rules (internal)
```

> *Note on naming: pipeline scripts have numbered filenames (e.g. `4. process_selection.py`). This is intentional — the number reflects the pipeline stage. Helper modules use clean names (`feature_graph.py`, `geometry_utils.py`) which are shims over the numbered source files.*

---

6. Local Development Setup

Prerequisites

- Miniconda with the `occ` environment (see `environment.yml`)
- Node.js 18+

Backend

```
# From repo root
cd capp_service
pip install -r requirements.txt
uvicorn main:app --reload --port 8001
```

Frontend

```
cd capp-frontend
npm install
```

```
npm start        # Starts at http://localhost:3000
```

Run a pipeline job manually (for testing pipeline changes)

```
conda run -n occ python "Claude output for program sheet/10. run_pipeline.py" "<path-to-part.step>"
```

---

7. The Pipeline — Stage Map

Each stage is a discrete Python script. Stages communicate through JSON files written to the job directory. No stage modifies the output of a previous stage.

Stage	Script	Takes	Produces
1	`extract_features.py`	`.step` file	`*_features.json`
2	`cluster_features.py`	features JSON	`*_clustered.json`
3	`classify_features.py`	clustered JSON	`*_classified.json`
4	`process_selection.py`	classified JSON	`*_processes.json`
5	`setup_planning.py`	processes JSON	`*_setups.json`
6	`tool_selection.py`	setups JSON	`*_tools.json`
7	`parameter_calculation.py`	tools JSON	`*_params.json`
8	`program_sheet.py`	params JSON	`*_program_sheet.pdf`

What each stage does at a high level:

- **Extract:** Parses the STEP file using OpenCASCADE. Outputs every face with its geometric type, orientation, area, and adjacency relationships between faces.
- **Cluster:** Groups adjacent faces into logical feature clusters based on geometric connectivity.
- **Classify:** Assigns a manufacturing feature type to each cluster (e.g. through hole, pocket, chamfer, boss, planar face). ~25 possible feature classes.
- **Process selection:** Maps each feature's type and dimensions to an ordered sequence of machining operations.
- **Setup planning:** Groups features by the direction they need to be machined from. Produces machine setups with workholding configuration, WCS origin, and stock state per setup.
- **Tool selection:** Matches each operation to a specific tool from the tool database.
- **Parameter calculation:** Computes spindle speed, feed rate, and depth-of-cut for every operation.
- **Program sheet:** Generates a PDF document summarising the full machining plan — one page per setup, with operation tables, tool list, WCS, and workholding.

---

8. Fixture Design Module

Located at `Claude output for program sheet/fixture_design/`. This module extends the pipeline output — it takes the setup planning results and generates:

- A fixture specification JSON (workholding type, jaw dimensions, clamp positions)
- Soft jaw STEP files for each setup that uses a vise (left jaw + right jaw)
- A fixture summary page appended to the program sheet PDF

The soft jaw geometry is computed from the part's bounding box and setup datum. The jaws are produced as parametric STEP solids that can be sent directly to a machine shop for machining.

---

9. Frontend — Pages and Tabs

Upload page (`/`)

User selects a `.step` / `.stp` file, enters a part name and material, and submits. Creates a job via `POST /jobs` and redirects to the viewer.

CappViewer (`/parts/:jobId`)

Three-panel layout: left sidebar (job list), centre (3D viewer iframe), right panel (analysis tabs).

Right panel tabs:

Tab	What it shows
Overview	Part dimensions, feature count, setup count, analysis warnings
Strategy	Machining setups list — workholding type, WCS, stock state, operations per setup
Program	Embedded PDF of the program sheet
Fixture	3D assembly view of the part + soft jaws for a selected setup
Features	Per-feature inspection data mapped to face indices
Closed Loop	Feedback capture from shop floor
Datums	Datum reference frame annotation tool
Inspection	Inspection checklist per feature
Rules	Validated machining rules backing the process decisions
AI Feed	AI-generated insights and alerts

---

10. Backend API — Route Summary

All routes served at `http://localhost:8001` in development.

Method	Route	Purpose
`POST`	`/jobs`	Upload STEP file, start pipeline
`GET`	`/jobs/{id}`	Poll job status and stage progress
`GET`	`/jobs/{id}/output/{stage}`	Retrieve JSON output for a specific stage
`GET`	`/jobs/{id}/pdf`	Download the program sheet PDF

GET	/jobs/{id}/step	Download the original STEP file
GET	/jobs/{id}/fixture	Get fixture specification JSON
GET	/jobs/{id}/fixture_assembly/{setup_index}	Get assembled STEP (part + jaws) for a setup
GET	/jobs/{id}/faces	Per-face feature data for the viewer
GET/POST	/jobs/{id}/datums	Read/write datum annotations
GET/POST	/jobs/{id}/tolerances	Read/write tolerance annotations

The job store is currently in-memory. Jobs are persisted to disk in `capp_service/jobs/{job_id}/` but the in-memory index is not recovered on service restart.

---

11. CAM Software Integration

ShiaanX is not a CAM tool — it sits above CAM. But for CNC programmers, their primary workspace is their CAM software. The integration goal is: ShiaanX generates the process plan, and the programmer finds it already pre-populated when they open their CAM tool — setups created, operations sequenced, tools assigned, parameters set. They verify, adjust where needed, and generate G-code. They are reviewing and approving, not creating from scratch.

What this means for the product: the CNC programmer becomes a reviewer of the AI-generated plan rather than the author of it. Their corrections are also valuable closed-loop data (see Section 12).

Starting point: Autodesk Fusion 360

Status: In progress. A Fusion 360 Python add-in exists in the `Toolpath/` directory of this repo. It is a working add-in that loads into Fusion 360 and provides a command panel. The codebase was adopted from an existing Fusion add-in foundation and extended with ShiaanX-specific commands.

ShiaanX commands built so far (in `Toolpath/code/commands/`):

Command	Purpose
command_ai_cam.py	End-to-end "Automated CAM" — submits part to ShiaanX and imports the resulting plan
command_setup_builder.py	Creates Fusion CAM setups from ShiaanX setup planning output
command_setup_rigger.py	Applies workholding configuration to a Fusion setup
command_make_softjaws.py	Imports soft jaw geometry into Fusion from the fixture module
command_import_workholding_files.py	Imports fixture STEP files into the Fusion assembly

What still needs wiring up:

- The add-in currently points at an external service URL. It needs to be redirected to the ShiaanX CAPP service (`localhost:8001` in development, the deployed service in production).
- Operation-level import (tools, feeds, speeds pre-populated) is partially built — the setup structure is further along than operation details.

Important constraints:

- Fusion's Python API has known limitations — some CAM operations require UI interaction that cannot be fully driven programmatically. Certain tool assignment flows hit this wall. Work on new operation types should validate API capability first before designing the full flow.
- G-code generation (post-processing) happens inside Fusion — ShiaanX does not generate G-code directly and never will.
- The add-in is a separate deployable from the web app. A CNC programmer installs it into Fusion 360 once; it then communicates with the ShiaanX backend over HTTP.

---

12. The Closed Loop — Core Concept

The closed loop is the central idea behind ShiaanX's long-term value, and understanding it is essential context for any work on this system.

The pipeline today generates a process plan from a CAD file. That plan goes to a CNC programmer who reviews it, makes any corrections, then programs the machine, cuts the part, and inspects it. Without a closed loop, ShiaanX is a static rule engine — useful, but not self-improving.

```
STEP file → Pipeline → Process Plan
                              │
                    CNC Programmer reviews
                    (corrections noted)
                              │
                      Part is machined
                              │
                      Part is inspected
                    (pass/fail, deviations)
                              │
                    Outcomes fed back in
                              ▼
                 Pipeline rules & models
                     get smarter
```

Data streams feeding the loop

There are several distinct data sources. Some are already live; others are planned.

1. CNC Controller Telemetry — time series (live)

A real CNC machine is already streaming telemetry into InfluxDB at ~1-second resolution. 20 fields are captured per sample:

Category	Fields
Axes	`axis_x`, `axis_y`, `axis_z`
Spindle	`spindle_speed`, `spindle_load`, `spindle_override`
Feed	`feed_rate`, `feed_override`
Program	`program_name`, `tool_number`, `block_number`
Timing	`cycle_time`, `cutting_time`, `program_runtime`, `production_time`

Machine state	`machine_state`, `machine_mode`, `alarm_active`, `production_count`

The `program_name` field encodes the part ID and setup sequence (e.g. `IR366 03104 00 1ST` = part IR366, drawing 03104, first setup). This is the key to linking a telemetry session back to a specific ShiaanX job and setup.

This data tells us: how long each setup actually took, what spindle loads the part generated, whether alarms fired, and at what point in the program things happened. This is ground truth that the pipeline's estimated parameters can be compared against.

2. CAM Files and G-code (via Fusion integration)

When a CNC programmer uses the Fusion add-in, they receive the ShiaanX process plan pre-populated. Whatever they change before generating G-code — different tool, adjusted feed rate, reordered operations, added a finishing pass — is captured in the Fusion CAM file (`.f3d`). The post-processed G-code is the final set of instructions that ran on the machine.

Together, CAM file + G-code give us:

- What the programmer accepted from the ShiaanX plan
- What they overrode, and what they changed it to
- The exact toolpath geometry and parameters that produced the part

This is the most direct signal for improving each pipeline stage: if the pipeline suggested a 2000 mm/min feed rate and every programmer changes it to 1500, that rule needs updating.

3. Business and order data — relational (live)

A PostgreSQL database on AWS tracks the full order lifecycle: enquiry receipt, part specifications, order status transitions, and delivery. This provides the business-level view of the loop — lead times, quote accuracy, on-time delivery.

4. Inspection results (planned)

There is currently no capture of whether a part passed or failed inspection, what feature failed, or how many rework cycles it took. This is the most critical missing layer. Without it, the loop cannot measure quality outcomes. Building the inspection capture is a prerequisite for computing meaningful quality metrics.

5. File storage

CAD files, drawings, and PDFs are stored in AWS S3. These are the static artefacts associated with each job.

What the loop enables once connected

Signal	Trains / improves
Controller cycle time vs. estimated	Parameter calculation (Stage 7)
Spindle load vs. expected	Tool selection and cutting parameters
Programmer's CAM corrections	Process selection, setup planning, tool selection
Inspection pass/fail per feature	Feature classification, process selection
Alarm events	Safety margins, depth-of-cut limits

Current state: The telemetry and business databases are live and being populated. The Fusion add-in bridge (for CAM file capture) is in progress. The inspection layer does not yet exist. The ShiaanX CAPP service does not yet read from any of these — connecting the data stores to the pipeline improvement cycle is the next major phase of work.

Why this matters architecturally: every tab in the viewer that looks like "just a display" is a data capture surface. The Datum Tagger, Inspection tab, and Closed Loop tab are all collection points feeding this infrastructure.

---

13. AI Model Improvement — The Major Technical Work

This is the highest-priority engineering work in the system right now, and it cuts across every pipeline stage.

The pipeline currently produces plans that are directionally correct but not yet at the level of an experienced machinist. The gaps are:

Stage	Current gap
Feature recognition (Stage 3)	Some feature types are not classified correctly, particularly complex pockets, slots, and angled features
Setup planning (Stage 5)	Setup groupings and workholding choices don't always match how a real shop would approach the part
Tool selection (Stage 6)	Tool choices are valid but not always optimal — an experienced programmer would pick differently
Parameter calculation (Stage 7)	Values are catalogue-derived starting points; actual shop-proven feeds and speeds differ meaningfully

The goal is to close these gaps progressively using a combination of:

- Validated rules derived from real shop audits (comparing pipeline output to what experienced machinists actually did)
- ML models trained on the growing dataset of processed parts
- Closed-loop feedback data from real production runs (see Section 12)

For a developer working on pipeline stages: each stage has a rule layer and will eventually have an ML layer alongside it. Work on either layer should be driven by concrete accuracy gaps measured against real parts, not theoretical improvements. The rule sheets in `rule_sheets/` document which rules have been validated against real shop data and which are still theoretical.

---

14. Development Conventions

Branch naming: `feature/<topic>` , `fix/<topic>` , `docs/<topic>`

Main branch: `main` — kept stable; features merged via PR

Commit style: Conventional commits (`feat:` , `fix:` , `docs:` , `refactor:`). Scope in parens where helpful: `feat(fixture-viewer):` .

Pipeline changes: Always test against at least one real STEP file end-to-end after modifying any pipeline script. The Motor Mount part in `capp_service/jobs/` (any recent job) is the reference test part.

Numbered files: Do not rename pipeline scripts. The numbering is load-bearing for readability and is referenced throughout internal documentation.

Frontend: No UI framework (no MUI, no Tailwind). Plain inline styles and CSS classes. Keep it consistent with the existing style — see `App.css`.

---

15. Key Technical Decisions

A few decisions worth knowing before you start touching things:

Tool database is metric, catalogue-sourced. All tooling data comes from published manufacturer catalogues (Sandvik Coromant / Kennametal) for aluminium. Do not substitute values from inch-based sources without explicit review — unit errors on cutting parameters can cause real damage on a machine.

Pipeline stages are stateless scripts, not a framework. Each stage reads its input JSON and writes its output JSON. There is no pipeline orchestration library — `runner.py` calls them as subprocesses in sequence. This keeps them independently testable.

Job store is in-memory. The `jobs: dict` in `main.py` is not persisted across service restarts. A database-backed store is on the roadmap but not yet implemented. Don't design features that depend on job history surviving a restart.

3D viewer is embedded via iframe. The viewer component is not a native React component — it's a URL that renders a CAD viewer. Communication between the viewer and the tab panels happens via `postMessage`.

For questions about pipeline logic or machining rules, speak to the lead engineer directly — those are not documented here.